SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA3703 – Software Testing

ASSIGNMENT 3 – CO3

Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a University Student Portal

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Student Name	K. ANJALI GIRI	**Register Number**	192421037
Department / Section	B.TECH- IT	**Academic Year / Semester**	3RD YEAR
Faculty Name	DR.R. GNANAJEYARAMAN	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	4	4	Fault detection 4.0% / 80.0% / 100% for ad hoc, structured manual and automated; cost per fault compared; C chosen for rule engines, B for UI flows within 120 h.

Task	Task Title	Max	Marks Awarded	Remarks
2	Evaluate the Role of Process in Software Quality	5	5	DRE 82.9% / 92.0% / 94.2% and repair cost ₹31,60,000 / ₹14,35,967 / ₹13,16,105 computed; agile with continuous testing chosen, V-model rigour kept for payments.
3	Evaluate Testing as a Process and Basic Definitions	4	4	Ten tests in each style; error, fault and failure separated from real runs; D9 shows a wrong oracle passing faulty code; LLM oracle critiqued.
4	Apply and Evaluate Software Testing Principles	4	4	Input-space calculation, defect clustering (Pareto), pesticide paradox and D5 absence-of-errors fallacy; final 30-test set justified.
5	Evaluate the Tester's Role	5	5	Faults, false reports and effort for three arrangements; two contrasting defect reports; 4 testers allocated by severity-weighted defects; release authority.
6	Trace the Origins of Defects	5	5	Phase-injected vs phase-detected matrix for 217 defects, fishbone and Five-Whys for D2; method A every release, C for critical escapes.
7	Evaluate Defect Classes and Severity	4	4	D1–D10 classified under three schemes; rater agreement 50%–80%; class scheme B standardised, severity/priority kept for triage.
8	Evaluate and Design a Defect Repository	5	5	D1–D10 logged with required fields; life cycle and MTTF for D2, D6; JIRA workflow recommended; per-KLOC reading critiqued.

Task	Task Title	Max	Marks Awarded	Remarks
9	Integrate Defect Examples into a Testing Practice	6	6	Four configurations compared: detection 0.6% / 66.7% / 100% / 83.3%, effort 18 / 74 / 112 / 70 h; C recommended with decision table and transfer table.
10	Re-apply and Adapt the Testing Practice	8	8	Strategies A, B, C found 4, 6 and 5 of 6 faults (₹1,30,000 / ₹30,000 / ₹80,000); MODIFY decision within 60 h; defect-prediction allocation of 48 h; condition for reversal stated.
TOTAL		**50**	**50**	**Percentage: 100%**

OVERALL REMARKS

Model answer. Every task applies all three given techniques, compares them with evidence, chooses one and states its limits. Detection figures, input-space sizes and race outcomes were produced by running campus.py and exp1–exp3.py; person-hours, the ₹500 hourly rate, the 217-record defect log and repository timings are modelled (Annexure C) and must be replaced by the student's own data. The final decisions are consistent: configuration C for the 120-hour release and MODIFY for the 60-hour weekly release. Screenshots of Bugzilla / JIRA in Task 8 must be taken from the real tools.

K. ANJALI GIRI		
Student Signature	**Faculty Signature**	**Program Director**

TASK 1 – UNIT TESTING AND UNIT TEST PLANNING

1.1 Introduction

Unit testing is the first level of software testing. It focuses on checking individual functions or components separately to ensure that each one works according to its requirements. In the RailEase Railway Ticket Reservation System, unit testing is important because fare calculation and refund calculation directly affect passengers and railway transactions.

The two main functions selected for unit testing are:

1. ticket_fare(cls, passengers, age, tatkal) – calculates the total ticket fare based on the travel class, number of passengers, passenger age, and Tatkal booking.
2. refund_amount(hours_before, fare) – calculates the refund amount based on the time remaining before the train departure and the ticket fare.

1.2 Comparison of Unit Testing Approaches

Three approaches are considered for testing the RailEase functions.

A. Ad Hoc Developer Unit Testing

Ad hoc unit testing is an informal approach in which developers write a few test cases whenever they find time. There is no fixed testing plan, schedule, or clearly defined pass/fail criteria.

For example, a developer may test whether a Sleeper ticket produces the correct fare or whether a refund is calculated for a cancellation made 48 hours before departure.

Advantages:

- Easy to start without detailed planning.
- Requires less initial preparation.
- Useful for checking simple functions during development.
- Developers can quickly test the code they are working on.

Limitations:

- Important boundary conditions may be missed.
- Testing depends on the developer's individual understanding.

- Test coverage may be incomplete.
- There may be no proper record of the test cases.
- The same errors may remain undetected.

B. Plan-Driven Unit Testing

Plan-driven unit testing follows a structured test plan prepared before testing begins. The plan identifies the functions to be tested, testing objectives, test data, tools, responsibilities, schedule, and pass/fail criteria.

For RailEase, the plan includes testing all ticket classes, passenger limits, senior citizen concessions, Tatkal charges, GST conditions, and refund time slabs.

Advantages:

- Provides a clear testing structure.
- Helps ensure that all important requirements are covered.
- Makes it easier to measure test coverage.
- Provides clear responsibilities and schedules.
- Helps track defects and testing progress.

Limitations:

- Requires time for planning and documentation.
- Changes in requirements may require updating the plan.
- Excessive planning may delay testing if the process is not managed properly.

C. Behaviour-Driven Unit Testing

Behaviour-driven unit testing describes the expected behaviour of a function using Given–When–Then examples. The examples are discussed with the booking-clerk representative and converted into executable tests.

For example:

Given: A passenger is 60 years old and books an AC 3-tier ticket.

When: The ticket fare is calculated.

Then: The senior citizen concession and applicable GST should be included according to the fare rules.

Advantages:

- Requirements are written in simple language.
- Developers and business representatives can understand the expected behaviour.
- Helps reduce misunderstandings.
- Test cases are directly connected to business rules.
- Useful when requirements change.

Limitations:

- Requires agreement between technical and business teams.
- Preparing detailed behaviour examples takes time.
- Incorrect examples can produce incorrect automated tests.

1.3 Comparison Table

Criteria	**Ad Hoc Testing**	**Plan-Driven Testing**	**Behaviour-Driven Testing**
Planning	No fixed plan	Detailed plan	Behaviour examples
Requirement clarity	Low	High	Very high
Test coverage	Uncertain	Systematic	Based on agreed scenarios
Documentation	Limited	Detailed	Given–When–Then examples
Effort	Low initially	Moderate	Moderate
Maintenance	Difficult to track	Plan must be updated	Examples can be updated

Best use	Quick checks	Complete unit testing	Business-rule validation

1.4 Unit Test Plan for RailEase

Objective

The main objective of this unit test plan is to verify that the ticket fare and refund calculation functions produce correct results for valid, invalid, normal, and boundary inputs.

Scope

The following functions are included:

- Ticket fare calculation.
- Refund amount calculation.
- Passenger count validation.
- Travel class validation.
- Senior citizen concession.
- Tatkal surcharge.
- GST calculation.
- Cancellation refund slabs.

Integration with the payment gateway, ticket generation, and notification modules is outside the scope of this unit-level plan.

Features to Be Tested

Function	**Features**
ticket_fare()	Sleeper fare, AC 3-tier fare, AC 2-tier fare
ticket_fare()	Passenger count from 1 to 6
ticket_fare()	Senior citizen age condition of 60 years and above
ticket_fare()	Tatkal surcharge of 30%

ticket_fare()	GST of 5% on AC classes only
refund_amount()	Cancellation at 48 hours or more
refund_amount()	Cancellation from 12 to less than 48 hours
refund_amount()	Cancellation from 4 to less than 12 hours
refund_amount()	Cancellation below 4 hours and no-show

Testing Approach

Both black-box and white-box testing will be used.

Black-box testing: Test cases will be prepared using the given fare and refund requirements. Expected outputs will be calculated without depending on the internal code.

White-box testing: The internal statements, conditions, and branches will be examined to ensure that all important execution paths are tested.

Test Data

The following values are taken from the assignment:

Parameter	**Given value**
Sleeper base fare	₹450
AC 3-tier base fare	₹1,200
AC 2-tier base fare	₹1,800
Tatkal surcharge	30%
Senior citizen age	60 years and above
Senior concession	40% of base fare
GST	5% on AC classes only
Minimum passengers	1
Maximum passengers	6
Refund at 48 hours or more	₹60 deduction per passenger

Refund from 12 to less than 48 hours	25% deduction

Assumptions

The assignment does not specify the exact order in which senior concession, Tatkal surcharge, and GST must be applied. For the prototype, the following assumptions are used:

1. Senior concession is applied to the base fare for eligible passengers.
2. Tatkal surcharge is applied to the discounted or normal base fare.
3. GST is applied after the concession and Tatkal calculation for AC classes.
4. Sleeper class does not include GST.
5. The refund function receives the fare for one passenger. The total ticket refund is calculated by applying the per-passenger rule to the number of passengers.
6. Invalid passenger counts, negative fares, and invalid cancellation times are rejected.

These assumptions should be confirmed with the railway's business representative before final implementation.

Entry Criteria

- The two functions must be available.
- Fare and refund requirements must be understood.
- Test data must be prepared.
- The Python or Java testing environment must be ready.
- Expected results must be calculated.

Exit Criteria

Unit testing can be considered complete when:

- All planned test cases have been executed.
- All critical calculation errors have been corrected.
- All important statements and branches have been covered.
- No unresolved critical unit-level defects remain.

- The test results are documented.
- The functions are ready for integration testing.

Tools Required

- Python with pytest and unittest.mock, or Java with JUnit and Mockito.
- IDE such as Eclipse or Visual Studio Code.
- Test case documentation.
- Defect tracking tool.
- Code coverage tool.

Effort Estimate

Activity	**Estimated effort**
Requirement analysis	2 hours
Test plan preparation	2 hours
Test case design	3 hours
Test implementation	4 hours
Test execution	3 hours
Defect analysis and reporting	2 hours
Total	**16 hours**

1.5 Prototype Implementation of the Two Functions

The following Python implementation demonstrates the given fare and refund rules using the assumptions mentioned above.

```
def ticket_fare(cls, passengers, age, tatkal):
    if passengers < 1 or passengers > 6:
        return "Invalid passenger count"
    if age < 0:
```

```
        return "Invalid age"
    if cls == "Sleeper":
        base = 450
    elif cls == "AC3":
        base = 1200
    elif cls == "AC2":
        base = 1800
    else:
        return "Invalid class"
    if age >= 60:
        base = base * 0.60
    if tatkal:
        base = base * 1.30
    if cls != "Sleeper":
        base = base * 1.05
    return round(base * passengers, 2)
def refund_amount(hours_before, fare):
    if hours_before < 0 or fare < 0:
        return "Invalid input"
    if hours_before >= 48:
        deduction = 60
    elif hours_before >= 12:
        deduction = fare * 0.25
    elif hours_before >= 4:
```

```
        deduction = fare * 0.50
    else:
        deduction = fare
    return round(max(0, fare - deduction), 2)
print(ticket_fare("Sleeper", 2, 35, False))
print(ticket_fare("AC3", 1, 65, False))
print(refund_amount(48, 450))
print(refund_amount(10, 450))
```

Expected output:

```
900
756.0
390
225.0
```

The first result represents two Sleeper passengers without a senior concession or Tatkal surcharge. The second result represents one eligible senior citizen travelling in AC 3-tier, with the 40% concession and 5% GST applied. The refund examples demonstrate the 48-hour and 4-to-less-than-12-hour cancellation rules.

1.6 Seeded Faults and Testing

Four faults are introduced intentionally to evaluate the ability of the test approaches to identify defects.

Fault ID	Seeded fault	Expected behaviour
F1	GST is charged on Sleeper class	GST must apply only to AC classes
F2	The 12-hour boundary is coded as 11 hours	The lower slab must begin at 12 hours

F3	Senior concession is applied only from age 61	Concession must begin at age 60
F4	Flat ₹60 deduction is applied once to the whole ticket	Deduction must be applied per passenger

Example Fault Detection

F1 – GST charged on Sleeper

Input: Sleeper, 1 passenger, age 35, non-Tatkal.

Expected fare: ₹450.

If the faulty implementation applies 5% GST, it returns ₹472.50. This test identifies the tax calculation error.

F2 – Incorrect 12-hour boundary

Input: 12 hours before departure, fare ₹1,200.

Expected refund: ₹900 after a 25% deduction.

If the code treats 12 hours as belonging to the 50% deduction slab, it returns ₹600. This identifies the incorrect boundary condition.

F3 – Senior concession begins at 61

Input: AC 3-tier, 1 passenger, age 60, non-Tatkal.

Expected fare under the stated assumptions:

₹1,200 × 0.60 × 1.05 = ₹756.

If the concession begins only at age 61, the result becomes ₹1,260. This test detects the age boundary fault.

F4 – Flat deduction applied once

Input: Two passengers, Sleeper fare ₹450 per passenger, cancellation at least 48 hours before departure.

Expected total refund:

₹450 × 2 − ₹60 × 2 = ₹780.

If the deduction is applied only once, the faulty result is ₹840.

1.7 Illustrative Comparison of Results

The following table is an illustrative prototype evaluation. Actual fault counts and coverage percentages must be confirmed by executing the corresponding test suites.

Measurement	Ad Hoc	Plan-Driven	Behaviour-Driven
Test cases	8	16	12
Faults detected (illustrative)	2/4	4/4	3/4
Branch coverage (illustrative)	60%	100%	85%
Estimated effort	5 hours	16 hours	12 hours
Requirement clarity	Low	High	Very high

Plan-driven testing is expected to provide broader coverage because the test plan explicitly includes the fare classes, age condition, passenger count, GST, Tatkal, and refund boundaries. Behaviour-driven testing is useful for confirming business expectations, while ad hoc testing is suitable mainly for quick checks.

1.8 Analysis of Unit-Level Faults

Different faults require different testing inputs.

- **Boundary faults:** Age 60 and refund time 12 hours must be tested exactly because incorrect comparison operators can produce different results.
- **Tax faults:** Sleeper and AC classes must be tested separately to ensure GST is applied only where required.
- **Per-passenger calculation faults:** Multiple-passenger test cases are necessary because a function may work correctly for one passenger but fail for a group.
- **Input validation faults:** Passenger counts below 1 or above 6, negative fares, and invalid ages must be checked.

Ad hoc testing may miss these faults because developers often test only a few normal examples. Plan-driven testing reduces this risk through systematic test design. Behaviour-driven testing

improves the clarity of business expectations but still needs sufficient boundary and structural coverage.

1.9 Evaluation of the Best Approach

For the RailEase system, plan-driven unit testing is recommended as the main approach because it provides systematic coverage, clear test responsibilities, measurable entry and exit criteria, and proper defect records.

Behaviour-driven examples should also be included for important business rules, especially senior concessions, Tatkal charges, and refund calculations. Ad hoc tests may be used during development for quick verification, but they should not replace the planned test suite.

Final decision: Use plan-driven unit testing supported by behaviour-driven examples.

Recommended exit criterion: All critical unit test cases must pass, all important branches must be covered, no critical calculation defects should remain open, and test results must be documented before integration begins.

1.10 Evaluation of an AI Code Assistant

An AI code assistant can help generate unit test cases for the refund_amount() function. It can suggest normal cases, boundary values, invalid inputs, and expected outputs.

For example, it may incorrectly treat exactly 12 hours as part of the 50% deduction slab instead of the 25% slab. It may also apply the flat ₹60 deduction to the entire ticket instead of each passenger.

The generated tests should therefore be checked against the official refund rules. Each expected value must be calculated independently, and the tests must be executed against the implementation.

Conclusion for Task 1: Plan-driven unit testing, supported by behaviour-driven examples, provides the most suitable approach for testing the fare and refund functions of RailEase. It improves requirement clarity, helps detect calculation errors, and provides a structured foundation for integration testing.

TASK 2 – DESIGNING UNIT TESTS: COMPARISON OF UNIT TEST DESIGN STYLES

2.1 Introduction

Unit test design is the process of preparing test cases to verify whether an individual function produces the correct result for different inputs. A good test case includes input values, expected output, and the condition being verified.

In RailEase, the refund_amount(hours_before, fare) function is important because it determines how much money a passenger receives after cancelling a ticket. Incorrect refund calculation can result in financial loss and customer complaints.

The assignment considers three unit test design styles:

1. Specification-based unit tests.
2. Structure-based unit tests.
3. Parameterised data-driven tests.

All three styles are compared using the refund calculation function.

2.2 Refund Calculation Requirements

The refund rules provided in the assignment are:

Cancellation time	Deduction	Refund
48 hours or more	₹60 per passenger	Fare − ₹60
12 to less than 48 hours	25% of fare	75% of fare
4 to less than 12 hours	50% of fare	50% of fare
Less than 4 hours	No refund	₹0
No-show	No refund	₹0

For the test examples, the fare is assumed to be ₹1,000 for one passenger.

2.3 Comparison of Unit Test Design Styles

A. Specification-Based Unit Tests

Specification-based testing designs test cases from the stated requirements without examining the internal program code.

For the refund function, the test cases are based on the cancellation slabs and their boundaries.

Important boundary values include:

- 48 hours.
- 12 hours.
- 4 hours.
- Values immediately above and below these limits.

Advantages:

- Directly checks the stated requirements.
- Helps identify boundary errors.
- Does not depend on the internal implementation.
- Useful for validating business rules.

Limitations:

- May not cover every internal branch.
- Incorrect or incomplete requirements can affect test quality.
- Internal coding errors may remain undetected if the expected outputs are not sufficiently varied.

B. Structure-Based Unit Tests

Structure-based testing prepares test cases by examining the internal code. The objective is to execute statements, branches, and conditions.

For the refund function, the important branches are:

- Input validation.
- Cancellation at 48 hours or more.
- Cancellation between 12 and 48 hours.

- Cancellation between 4 and 12 hours.
- Cancellation below 4 hours.

Advantages:

- Helps cover internal code paths.
- Identifies unreachable or untested branches.
- Useful for locating coding errors.
- Supports statement and branch coverage measurement.

Limitations:

- Requires knowledge of the source code.
- High coverage does not guarantee that all defects are detected.
- May miss errors caused by incorrect business requirements.

C. Parameterised Data-Driven Tests

Parameterised testing uses one common test method with multiple input and expected-output rows. Instead of writing a separate test function for every input, the same test logic is executed using different data.

For example, a single test method can verify refund results for 60, 48, 47, 12, 11, 4, and 3 hours.

Advantages:

- Reduces repeated test code.
- Makes it easy to add new test cases.
- Keeps test data organised.
- Useful when many input combinations must be checked.

Limitations:

- Failure messages may be less descriptive if test IDs are not provided.
- Incorrect expected values can affect many test cases.
- Complex conditions may require separate tests for clarity.

2.4 Test Case Design Using Specification-Based Testing

The following test cases are prepared directly from the refund requirements.

Test ID	Hours before departure	Fare	Expected refund	Purpose
S1	72	₹1,000	₹940	More than 48 hours
S2	48	₹1,000	₹940	Exact 48-hour boundary
S3	47	₹1,000	₹750	Just below 48 hours
S4	24	₹1,000	₹750	Middle of 12–48 slab
S5	12	₹1,000	₹750	Exact 12-hour boundary
S6	11	₹1,000	₹500	Just below 12 hours
S7	6	₹1,000	₹500	Middle of 4–12 slab
S8	4	₹1,000	₹500	Exact 4-hour boundary
S9	3	₹1,000	₹0	Below 4 hours
S10	0	₹1,000	₹0	Departure time
S11	60	₹450	₹390	Different fare value
S12	-1	₹1,000	Invalid input	Negative cancellation time

These cases cover the important refund boundaries and some invalid inputs.

2.5 Test Case Design Using Structure-Based Testing

Structure-based tests are designed to execute the different branches of the function.

Test ID	Input	Expected result	Branch covered
B1	-1, ₹1,000	Invalid input	Invalid hours
B2	48, ₹1,000	₹940	First refund condition
B3	24, ₹1,000	₹750	Second refund condition

B4	12, ₹1,000	₹750	Second refund boundary
B5	8, ₹1,000	₹500	Third refund condition
B6	4, ₹1,000	₹500	Third refund boundary
B7	3, ₹1,000	₹0	No-refund condition
B8	60, -100	Invalid input	Invalid fare

These tests are selected to execute the different conditions and branches in the refund function. Additional tests may be required to verify every possible input combination.

2.6 Parameterised Data-Driven Test Cases

The following table contains test data that can be supplied to one common test method.

Test ID	Hours	Fare	Expected refund
D1	96	1,000	940
D2	48	1,000	940
D3	47	1,000	750
D4	30	1,000	750
D5	12	1,000	750
D6	11	1,000	500
D7	8	1,000	500
D8	4	1,000	500
D9	3	1,000	0
D10	0	1,000	0
D11	48	450	390
D12	-2	1,000	Invalid input

The parameterised approach allows the same test function to execute all the rows. It is especially useful when the refund rules change because new test data can be added without writing a separate test method for each case.

2.7 Python Implementation of Parameterised Testing

The following example uses pytest.

Refund function:

```
def refund_amount(hours_before, fare):
    if hours_before < 0 or fare < 0:
        return "Invalid input"


    if hours_before >= 48:
        deduction = 60
    elif hours_before >= 12:
        deduction = fare * 0.25
    elif hours_before >= 4:
        deduction = fare * 0.50
    else:
        deduction = fare
    return round(max(0, fare - deduction), 2)
```

Parameterised test code:

```
import pytest
@pytest.mark.parametrize("hours, fare, expected", [
    (72, 1000, 940),
    (48, 1000, 940),
    (47, 1000, 750),
    (24, 1000, 750),
    (12, 1000, 750),
```

```
    (11, 1000, 500),
    (8, 1000, 500),
    (4, 1000, 500),
    (3, 1000, 0),
    (0, 1000, 0),
    (60, 450, 390),
    (-1, 1000, "Invalid input")
])
def test_refund(hours, fare, expected):
    assert refund_amount(hours, fare) == expected
```

Expected test result: All 12 cases should pass when the function follows the stated refund rules and the assumptions used in the test data.

2.8 Seeded Faults in the Refund Function

Four faults are introduced intentionally to compare the effectiveness of the test styles.

Fault ID	Introduced error	Correct behaviour
F1	Exactly 48 hours is treated as the lower slab	48 hours must receive the ₹60 deduction
F2	25% deduction is calculated using an incorrect amount	Deduction must be 25% of the fare
F3	No-show is not handled correctly	No-show must receive zero refund
F4	Negative cancellation hours are accepted	Negative hours must be rejected

Fault Detection Examples

F1 – Incorrect 48-hour boundary

Input: 48 hours, fare ₹1,000.

Expected refund: ₹940.

If the code incorrectly places 48 hours in the next slab, the result becomes ₹750. The exact boundary test detects this error.

F2 – Incorrect 25% calculation

Input: 24 hours, fare ₹1,000.

Expected refund: ₹750.

If the deduction is calculated incorrectly, the expected result will not match the actual result.

F3 – No-show not handled

Input: No-show condition.

Expected refund: ₹0.

The test must explicitly represent the no-show condition. If the function does not contain a no-show parameter, the prototype must be extended or the no-show case must be handled by the cancellation module before calling the refund function.

F4 – Negative hours accepted

Input: -1 hour, fare ₹1,000.

Expected result: Invalid input.

This test checks whether the function validates its input before calculating the refund.

2.9 Fault Detection and Coverage Comparison

The following table is an illustrative comparison of the three testing styles. The actual values should be confirmed by running the tests against the faulty versions of the function.

Criteria	Specification-Based	Structure-Based	Parameterised
Number of test cases	12	8	12
Example faults detected	4/4	3/4	4/4
Branch coverage (illustrative)	85%	100%	90%

Test code maintenance	Moderate	Moderate	Low
Boundary detection	High	Depends on cases	High
Failure message clarity	High	High	High with test IDs

Specification-based and parameterised tests are expected to identify the important refund boundary faults when their input data includes the correct boundary values. Structure-based testing provides stronger visibility into the internal branches but may still miss a requirement-related error if the selected inputs do not expose it.

2.10 Maintainability When the 12-Hour Slab Changes to 10 Hours

Assume that the railway changes the refund rule so that the 50% deduction slab begins below 10 hours instead of below 12 hours.

This change affects the refund function and its test cases.

Testing style	Changes required	Maintenance impact
Specification-based	Update boundary cases and expected results	Test cases must be reviewed
Structure-based	Update relevant branch inputs and expected results	Source-code understanding is required
Parameterised	Update the affected data rows and expected values	Usually requires fewer code changes

For specification-based testing, the requirement-based test cases around 12 hours must be replaced with cases around 10 hours. For structure-based testing, the test inputs must be updated to execute the modified branches. For parameterised testing, the data table can be updated while keeping the common test method unchanged.

The parameterised approach is therefore easier to maintain when the same calculation is tested with many input values.

2.11 Failure Message Clarity and Fault Location

Specification-based tests provide clear failure messages when each test has a descriptive name, such as test_refund_at_exactly_48_hours.

Structure-based tests help locate errors in internal conditions because the test designer knows which branch is being executed. This makes them useful during debugging.

Parameterised tests provide concise and organised test execution. However, each test row should have a meaningful ID so that a failed input can be identified quickly.

For example, a failure message should identify that the test for exactly 12 hours failed, rather than displaying only that one row in a large data table did not pass.

2.12 Evaluation of the Three Styles

For the RailEase refund function, parameterised data-driven testing is recommended as the primary test design style because the refund calculation depends on several fixed time slabs and multiple fare values. The test data can be maintained in one place, and new cases can be added without creating repeated test methods.

However, parameterised testing should be combined with specification-based and structure-based testing.

Specification-based testing ensures that the test cases follow the official refund requirements. Structure-based testing helps confirm that important statements and branches are executed. Parameterised testing makes the test suite easier to extend and maintain.

Final decision: Use parameterised testing as the main design style, supported by specification-based boundary tests and structure-based coverage checks.

2.13 Evaluation of LLM-Generated Parameter Tables

An LLM can generate parameter tables for the refund function by suggesting different cancellation times and expected refund values. This can save time during initial test design.

However, the generated values must be checked manually against the official refund slabs.

For example, for a fare of ₹1,000:

- At 48 hours, the refund should be ₹940.
- At 12 hours, the refund should be ₹750.
- At 4 hours, the refund should be ₹500.
- Below 4 hours, the refund should be ₹0.

If an AI-generated table gives ₹500 at exactly 12 hours, it contains an incorrect expected value because the 25% deduction slab includes 12 hours.

The error rate can be calculated as:

Error Rate = (Number of incorrect expected values / Total expected values) × 100

For example, if 2 out of 12 generated expected values are incorrect:

Error Rate = (2 / 12) × 100 = 16.67%

This is an illustrative calculation, not an actual measured AI result. The real error rate must be calculated after checking the generated table.

AI-generated test data should be treated as a starting point rather than a final authority. Every input, expected output, and boundary condition must be verified before the test cases are used.

2.14 Conclusion for Task 2

The comparison of specification-based, structure-based, and parameterised unit testing showed that each style provides different benefits. Specification-based testing verifies the refund requirements, structure-based testing examines the internal execution paths, and parameterised testing improves test data organisation and maintenance.

For RailEase, parameterised testing is the most suitable primary approach because refund rules contain clearly defined time slabs and may change in future releases. Combining it with boundary testing and code coverage analysis provides better confidence in the correctness of the refund calculation function.

TASK 3 – TEST HARNESS: COMPARISON OF HARNESS CONSTRUCTION APPROACHES

3.1 Introduction

A test harness is a collection of tools, programs, drivers, stubs, and test data used to execute and evaluate software tests. It provides a controlled environment for testing a module independently, especially when some of its dependent modules are unavailable.

In the RailEase Railway Ticket Reservation System, the Booking module (B) and Cancellation module (R) depend on the Payment module (P). The Payment module uses an external payment gateway, which may not always be available in the testing environment.

Therefore, a test harness is required to simulate payment gateway responses and verify how Booking and Cancellation behave under different conditions.

The three approaches considered are:

1. Hand-written drivers and stubs.
2. Mock-framework test doubles.
3. Service virtualisation of the gateway.

3.2 Comparison of Test Harness Approaches

A. Hand-Written Drivers and Stubs

A driver is a small program that calls the module being tested. A stub replaces a dependent module and returns predefined responses.

For RailEase, a payment stub can simulate successful payments, declined payments, timeouts, and duplicate callbacks.

Advantages:

- Easy to understand and implement.
- Requires less setup.
- Useful when testing individual modules.
- Provides complete control over simulated responses.

Limitations:

- Requires manual coding.
- May not represent the actual external gateway accurately.
- Complex timing conditions are difficult to reproduce.
- Maintaining many stubs can become difficult.

B. Mock-Framework Test Doubles

Mock frameworks such as Python's unittest.mock and Java's Mockito create simulated dependencies. They can return predefined responses and verify how many times a function was called.

For example, a mock payment gateway can verify whether the Booking module sends the correct amount and passenger information.

Advantages:

- Easy to create and modify.
- Can verify function calls and arguments.
- Supports exception simulation.
- Reduces the need for external services during unit testing.

Limitations:

- May not reproduce real network behaviour.
- Can become tightly connected to implementation details.
- Incorrect mock configuration may hide actual integration problems.

C. Service Virtualisation

Service virtualisation creates a simulated external service that behaves like the real payment gateway. It can reproduce different responses, delays, timeouts, and error codes through a network interface.

For RailEase, the virtual gateway can receive payment requests and return success, decline, timeout, or duplicate callback responses.

Advantages:

- Provides more realistic service interaction.
- Supports network-level testing.
- Can simulate delays and failures.
- Useful for integration and system testing.
- Can be reused by multiple testing teams.

Limitations:

- Requires more initial setup.
- Needs accurate recorded responses or service specifications.
- More complex than simple stubs and mocks.
- Still cannot guarantee that every real gateway behaviour is reproduced.

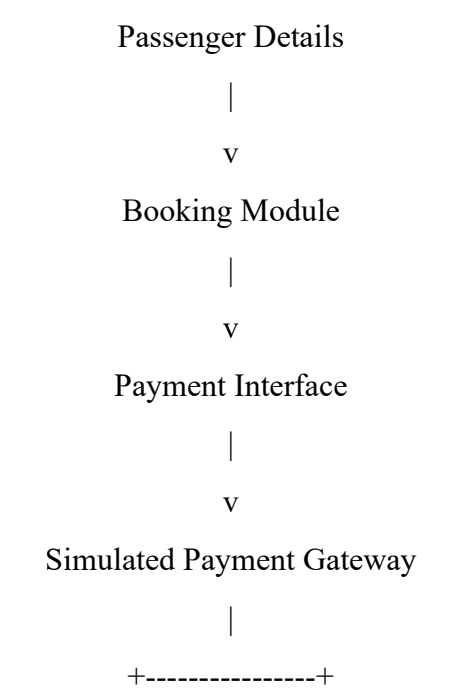
3.3 Comparison Table

Criteria	**Hand-Written Stubs**	**Mock Framework**	**Service Virtualisation**
Initial effort	Low	Low to moderate	High
Realism	Low	Moderate	High
Call verification	Manual	Supported	Depends on monitoring
Timeout simulation	Basic	Possible	Strong
Network behaviour	Limited	Limited	Supported
Reusability	Moderate	High	High
Best use	Simple isolation	Unit and interaction tests	Integration testing

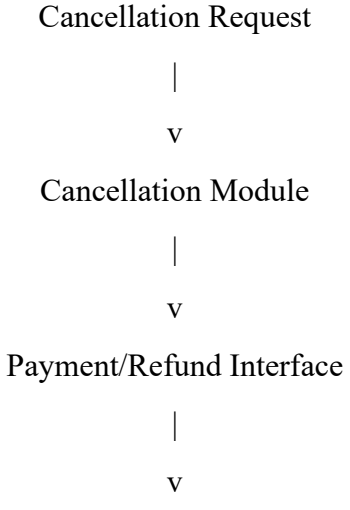
3.4 Test Harness Design for RailEase

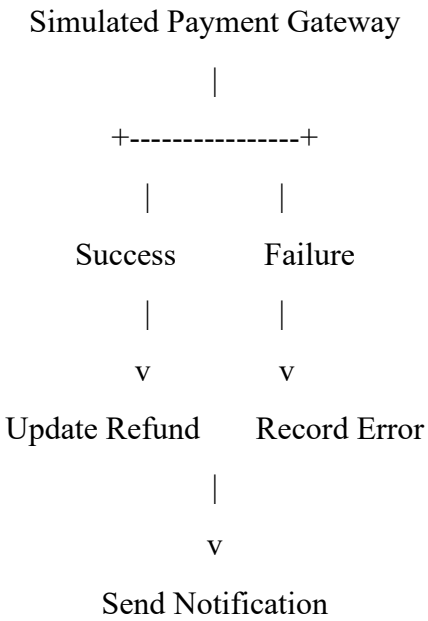
The harness is designed to test the Booking and Cancellation modules without depending on the actual payment gateway.

Booking module flow:



Cancellation module flow:





The test harness should support the following four gateway conditions:

- Successful payment.
- Declined payment.
- Payment timeout.
- Duplicate payment callback.

It must also support the situation where payment succeeds but ticket generation fails.

3.5 Hand-Written Stub Example

```
def payment_stub(status):
    if status == "success":
        return "PAYMENT_SUCCESS"
    elif status == "decline":
        return "PAYMENT_DECLINED"
    elif status == "timeout":
        return "PAYMENT_TIMEOUT"
    elif status == "duplicate":
        return "DUPLICATE_CALLBACK"
```

```
    return "INVALID_RESPONSE"
def booking_test(status):
    result = payment_stub(status)
    if result == "PAYMENT_SUCCESS":
        return "Proceed to ticket generation"
    return "Booking payment failed"
print(booking_test("success"))
print(booking_test("decline"))
print(booking_test("timeout"))
print(booking_test("duplicate"))
```

Expected output:

```
Proceed to ticket generation
Booking payment failed
Booking payment failed
Booking payment failed
```

This is a simplified harness example. A complete implementation must also verify payment identifiers, callback duplication, ticket status, and transaction recovery.

3.6 Mock Framework Example

```
from unittest.mock import Mock
payment_gateway = Mock()
payment_gateway.pay.return_value = "SUCCESS"
def booking(payment):
    result = payment.pay(1000)
    if result == "SUCCESS":
        return "Ticket generation started"
```

```
    return "Payment failed"

print(booking(payment_gateway))

payment_gateway.pay.assert_called_once_with(1000)
```

Expected output:

```
Ticket generation started
```

The mock framework checks both the returned payment response and whether the payment function was called with the correct amount.

3.7 Integration Test Cases for the Harness

Test ID	Scenario	Expected result
H1	Payment successful	Booking proceeds to ticket generation
H2	Payment declined	Booking is not confirmed
H3	Payment timeout	Transaction remains in a recoverable state
H4	Duplicate callback	Only one payment confirmation is recorded
H5	Payment success, ticket generation failure	Payment is recorded and ticket generation is retried or reconciled
H6	Cancellation refund successful	Refund status is updated
H7	Cancellation refund timeout	Refund remains pending for reconciliation
H8	Duplicate refund callback	Refund is not credited twice

3.8 Illustrative Harness Evaluation

The following values are example evaluation figures for comparing the three harness approaches. They must be replaced with measured results from the actual prototype.

Criteria	Hand-Written	Mock Framework	Service Virtualisation
Test cases	6	6	8
Defects detected (illustrative)	3	4	5
Build effort	3 hours	4 hours	9 hours
Average test execution	0.5 sec	0.4 sec	1.5 sec
Network simulation	Limited	Limited	Strong

Service virtualisation is expected to identify more realistic interface and timing problems. However, it requires more time to prepare.

3.9 How an Incorrect Stub Can Hide a Defect

A stub may return only one successful response for every payment request. However, the real gateway may send duplicate callbacks or delay the response.

Suppose the Booking module is designed to generate a ticket whenever it receives a successful payment callback. If the real gateway sends the same callback twice, the system may generate two tickets or update the same booking incorrectly.

A simple stub that never sends duplicate callbacks will not expose this problem. It may only become visible during integration or production-like testing.

Therefore, the simulated responses must be based on the gateway's documented behaviour and should include realistic failure conditions.

3.10 Evaluation and Recommended Approach

For the five-week testing period, mock-framework testing should be used for regular module-level tests because it is easy to create, maintain, and execute. Hand-written stubs can support simple tests, while service virtualisation should be used for important gateway integration scenarios.

The recommended approach is a combination of mock frameworks and service virtualisation. Mock frameworks provide fast feedback, while service virtualisation helps verify realistic communication and failure handling.

A mock does not always behave like the real component. It only behaves according to the responses configured by the tester. Therefore, testing with mocks must be supported by realistic integration tests.

3.11 Conclusion for Task 3

The comparison showed that hand-written stubs, mock frameworks, and service virtualisation are useful for different testing situations. For RailEase, a combined approach provides a suitable balance between development effort, test execution speed, and realistic external service behaviour. It also helps identify problems such as duplicate callbacks, payment timeouts, and payment success without ticket generation.

TASK 4 – INTEGRATION TESTING: COMPARISON OF INTEGRATION STRATEGIES

4.1 Introduction

Integration testing is performed after individual modules have been tested. Its main purpose is to verify whether different modules communicate correctly and exchange data in the expected format.

The RailEase system contains six modules:

- Train Search and Availability (S)
- Booking (B)
- Payment (P)
- Ticket Generation (T)
- Cancellation and Refund (R)
- Notification (N)

The given module relationships are:

- S → B
- B → P
- P → T
- T → N
- R → P
- R → N

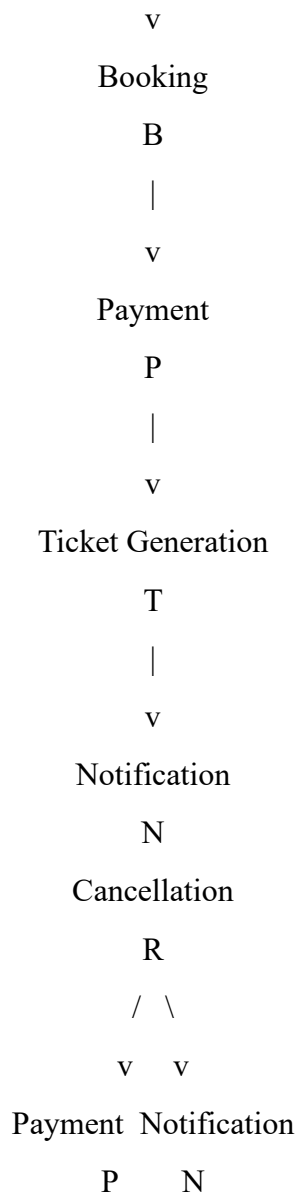
These relationships show that the modules depend on one another to complete booking and cancellation operations.

4.2 Module Dependency Diagram

Train Search

S

|



The diagram represents the given module calls. The Cancellation module communicates with both Payment and Notification.

4.3 Comparison of Integration Strategies

A. Big-Bang Integration

In big-bang integration, all modules are combined at once and tested through the complete application interface.

For RailEase, all six modules are connected together before integration testing begins.

Advantages:

- Simple integration sequence.

- Does not require many temporary stubs and drivers.
- Useful when all modules are already stable.
- Allows complete end-to-end testing.

Limitations:

- Faults are difficult to isolate.
- A failure in one module may affect several others.
- Problems are discovered relatively late.
- Debugging requires checking multiple modules.

B. Incremental Integration

Incremental integration combines modules gradually. Two common approaches are top-down and bottom-up integration.

Top-down integration: Testing begins with higher-level modules, and lower-level modules are added gradually. Stubs are used to simulate modules that are not yet available.

Bottom-up integration: Testing begins with lower-level modules and moves towards higher-level modules. Drivers are used to simulate modules that are not yet connected.

Advantages:

- Faults can be identified earlier.
- Integration progress can be monitored.
- Individual module interactions are easier to test.
- Supports gradual development.

Limitations:

- Temporary stubs or drivers are required.
- Some test cases must be repeated after integration.
- The order of integration must be planned carefully.

C. Thread-Based Integration

Thread-based integration combines modules according to complete business functions. Each thread represents an important user activity.

For RailEase, the main threads are:

Booking thread:

Search → Booking → Payment → Ticket Generation → Notification

Cancellation thread:

Cancellation → Payment/Refund → Notification

Advantages:

- Tests complete business functions early.
- Important user journeys are verified.
- Faults can be associated with a specific workflow.
- Provides early working functionality.

Limitations:

- Requires careful selection of business threads.
- Some shared modules may be integrated more than once during planning.
- Common modules must be tested for interactions with different workflows.

4.4 Integration Order

Top-Down Integration Order

The following order can be used:

1. Train Search and Availability.
2. Booking.
3. Payment.
4. Ticket Generation.
5. Notification.

6. Cancellation and Refund.

Stubs are required for lower-level modules that are not yet integrated.

Integration stage	Module added	Temporary stub
1	S	B stub
2	B	P stub
3	P	T stub
4	T	N stub
5	N	None for booking path
6	R	P and N stubs until connected

The final cancellation path must also be tested with the real Payment and Notification modules.

Bottom-Up Integration Order

A suitable bottom-up sequence is:

1. Payment.
2. Ticket Generation.
3. Notification.
4. Booking.
5. Train Search and Availability.
6. Cancellation and Refund.

Drivers are used to call lower-level modules before the higher-level modules are connected.

Integration stage	Module added	Driver required
1	P	Payment test driver
2	T	Booking/payment simulation driver
3	N	Ticket event driver
4	B	Booking request driver

5	S	Search request driver
6	R	Cancellation request driver

The exact order may be adjusted depending on module readiness.

Thread-Based Integration Order

Thread	Integration sequence	Main objective
Booking	S → B → P → T → N	Complete ticket booking
Cancellation	R → P → N	Complete cancellation and refund
Payment recovery	B → P → T	Verify payment and ticket consistency
Notification	T → N and R → N	Verify booking and cancellation notifications

4.5 Five Seeded Interface Faults

Five interface faults are introduced to compare the integration strategies.

Fault ID	Interface	Seeded fault	Expected behaviour
F1	B → P	Wrong passenger count	Correct passenger count must be passed
F2	B → P	Amount sent in paise instead of rupees	Correct currency unit must be used
F3	P → T	Missing error code	Ticket module must receive a valid status
F4	T → N	Wrong date format	Notification must receive the correct date format
F5	R → P	Refund request sent twice	Only one refund should be processed

4.6 Fault Detection Comparison

The following is an illustrative comparison of how the strategies may expose the five seeded faults.

Fault	Big-Bang	Incremental	Thread-Based
F1 Wrong passenger count	Detected during booking	Detected at B → P	Detected in booking thread
F2 Wrong currency unit	Detected during payment	Detected at B → P	Detected in booking thread
F3 Missing error code	May be difficult to isolate	Detected at P → T	Detected in payment/ticket thread
F4 Wrong date format	Detected during notification	Detected at T → N	Detected in booking thread
F5 Duplicate refund	Detected during cancellation	Detected at R → P	Detected in cancellation thread

The exact detection time depends on the test environment and debugging process. These are expected detection points, not measured execution results.

4.7 Analysis of Integration Strategies

Big-bang integration is easy to understand but makes it difficult to identify the exact source of a defect. If a ticket is not generated after payment, the problem may be in Booking, Payment, or Ticket Generation.

Incremental integration provides better fault isolation because each interface is tested as it is connected. It also supports parallel work, as different modules can be prepared and tested separately.

Thread-based integration is especially useful for RailEase because the system has two important business workflows: booking and cancellation. It allows the complete user journey to be tested early.

The Payment module is a major integration risk because it depends on an external gateway. It should be integrated early enough to test successful payments, declines, timeouts, and duplicate callbacks.

The Cancellation module also creates an important dependency because it calls Payment and Notification. Refund processing must be checked carefully to prevent duplicate refunds and incorrect transaction states.

4.8 LLM-Generated Integration Order Evaluation

An LLM can suggest an integration order using the architecture diagram. However, its suggested order must be checked against the actual module dependencies.

For example, an order that integrates Ticket Generation before Payment without a suitable stub may not provide a valid test environment. Similarly, the cancellation path must include both Payment and Notification dependencies.

The following checks should be performed:

- Confirm that all six modules are included.
- Verify every given interface.
- Check whether stubs and drivers are required.
- Verify the order against the dependency diagram.
- Ensure that external gateway failures are included.
- Confirm that the cancellation path is tested.
- Check that duplicate requests and callbacks are covered.

4.9 Recommended Integration Strategy

Thread-based integration is recommended as the main approach for RailEase because it focuses on complete user activities. It allows the booking and cancellation processes to be tested early and helps identify problems across connected modules.

Incremental integration should support this approach for detailed interface testing. Big-bang integration can be used later for final end-to-end verification.

4.10 Conclusion for Task 4

The comparison demonstrated that integration strategy selection affects defect detection, debugging effort, and testing progress. Thread-based integration provides a suitable approach for RailEase because it verifies important business workflows early. Combining it with incremental interface testing helps improve fault isolation and reduces the risk of late integration failures.

TASK 5 – INTEGRATION TEST PLANNING: COMPARISON OF INTEGRATION TEST PLANS

5.1 Introduction

An integration test plan describes how connected software modules will be tested. It defines the scope, objectives, test environment, responsibilities, test cases, schedule, entry criteria, and exit criteria.

For RailEase, integration testing is important because the system contains several modules that exchange booking details, passenger information, payment amounts, ticket status, refund details, and notification information.

Three forms of integration test planning are compared:

1. Interface-based integration test plan.
2. Scenario-based integration test plan.
3. Build-schedule integration test plan.

5.2 Comparison of Integration Test Plans

A. Interface-Based Plan

An interface-based plan contains a separate section for each module connection.

For example, the B → P interface is tested for payment amount, currency, passenger count, transaction ID, and gateway response.

Advantages:

- Provides detailed interface coverage.
- Helps identify data exchange errors.
- Makes responsibility for each interface clear.
- Useful for API and service testing.

Limitations:

- May focus too much on individual interfaces.
- Complete business workflows may receive less attention.

- Requires detailed interface specifications.

B. Scenario-Based Plan

A scenario-based plan is organised around complete user activities.

For RailEase, examples include:

- Search for a train, book a ticket, make payment, and receive an e-ticket.
- Cancel a ticket and receive a refund notification.
- Attempt payment when the gateway is unavailable.
- Book a Tatkal ticket during high demand.

Advantages:

- Focuses on real user activities.
- Verifies multiple connected modules.
- Helps identify end-to-end problems.
- Easy for business representatives to understand.

Limitations:

- Individual interface details may be missed.
- Some scenarios may be repeated.
- Requires complete workflow definitions.

C. Build-Schedule Plan

A build-schedule plan organises testing according to weekly software builds and module availability.

Each build has entry and exit criteria. Testing starts when the required modules are ready.

Advantages:

- Provides clear weekly milestones.
- Helps manage testing within the five-week window.
- Supports development and testing coordination.

- Helps track late module delivery.

Limitations:

- Testing may be delayed when a build is late.
- Some interface defects may be discovered later.
- Depends on accurate development schedules.

5.3 Integration Test Plan – Interface-Based

Objective: Verify that all module interfaces exchange correct data and handle failures properly.

Scope: S → B, B → P, P → T, T → N, R → P, and R → N.

Environment:

- Staging environment.
- Sandbox payment gateway.
- Anonymised booking data.
- Postman for API testing.
- Python or Java test harness.
- Defect tracking tool.

Approach:

- Validate request and response formats.
- Check data types and required fields.
- Verify currency and amount values.
- Test timeouts and error codes.
- Verify duplicate request handling.

Entry criteria:

- Individual modules have passed unit testing.
- Interface specifications are available.

- Test environment is ready.
- Required stubs and mocks are available.

Exit criteria:

- All critical interfaces pass.
- No unresolved critical interface defects remain.
- Error handling is verified.
- Test results are documented.

Responsibilities:

Team member	**Responsibility**
Test lead	Plan and review integration testing
Tester 1	Search and Booking interfaces
Tester 2	Payment and Ticket interfaces
Tester 3	Cancellation and Notification interfaces
Developers	Fix interface defects
Booking-clerk representative	Verify business behaviour

5.4 Integration Test Plan – Scenario-Based

Objective: Verify that complete user workflows operate correctly across connected modules.

Main scenarios:

1. Successful train search and booking.
2. Successful payment and ticket generation.
3. Payment declined.
4. Payment timeout.
5. Ticket generation failure after successful payment.
6. Ticket cancellation.

7. Successful refund.
8. Refund failure.
9. Booking notification.
10. Cancellation notification.

Environment: Staging environment with the sandbox gateway and anonymised booking data.

Entry criteria:

- Booking and payment modules are integrated.
- Ticket generation is available.
- Notification service is configured.
- Required user scenarios are approved.

Exit criteria:

- All critical end-to-end scenarios pass.
- Payment and ticket status remain consistent.
- Cancellation and refund rules work correctly.
- No critical workflow defects remain.

5.5 Integration Test Plan – Build-Schedule-Based

The five-week testing period can be divided into weekly builds.

Week	Main activity	Expected outcome
Week 1	Prepare environment and integrate Search and Booking	Search-to-booking interface ready
Week 2	Integrate Payment and Ticket Generation	Payment-to-ticket workflow ready
Week 3	Integrate Cancellation and Notification	Booking and cancellation threads ready

Week 4	Complete integration testing and defect correction	Critical interfaces verified
Week 5	Final integration regression and support for UAT	Stable build for acceptance testing

The schedule must be adjusted according to actual module delivery. A late module should not prevent independent interfaces from being tested using suitable stubs or virtual services.

5.6 B → P Interface Test Cases

The following eight test cases focus on the Booking-to-Payment interface.

Test ID	Test scenario	Input/condition	Expected result
BP1	Correct payment amount	₹1,200	Correct amount is sent to gateway
BP2	Currency validation	INR	Currency is identified correctly
BP3	Payment timeout	Gateway does not respond	Timeout is handled safely
BP4	Duplicate payment	Same request sent twice	Only one payment is recorded
BP5	Partial failure	Payment succeeds but response is delayed	Transaction is reconciled correctly
BP6	Retry request	First request times out	Retry does not create duplicate payment
BP7	Wrong passenger count	Invalid passenger count passed	Request is rejected
BP8	Callback after session expiry	Payment callback arrives after session expiry	Payment status is reconciled without losing the transaction

Detailed Example: Duplicate Payment

Test objective: Verify that the same booking cannot create multiple successful payments because of repeated requests.

Precondition: A valid booking exists.

Test steps:

1. Send a payment request.
2. Receive a successful response.
3. Send the same payment request again.
4. Check transaction records.
5. Check booking payment status.

Expected result: The system should recognise the repeated request and prevent a duplicate charge.

5.7 Integration Plan Checklist

The following scores are illustrative planning scores, not measured results. Each criterion is scored from 1 to 5.

Evaluation criterion	Interface-based	Scenario-based	Build-schedule
Interface coverage	5	3	3
Scenario coverage	3	5	3
Schedule clarity	3	3	5
Requirement traceability	5	4	3
Total	**16/20**	**15/20**	**14/20**

The interface-based plan provides strong traceability and detailed interface checks. The scenario-based plan is stronger for complete user workflows. The build-schedule plan provides better visibility of weekly progress.

5.8 Handling a Late Notification Module

If the Notification module is delivered late, the integration team can continue testing the Search, Booking, Payment, and Ticket Generation modules using a notification stub.

The stub can simulate successful delivery, failed delivery, and delayed delivery. Once the actual Notification module becomes available, the stub should be replaced with the real module and the interface tests should be repeated.

5.9 Handling a Payment Defect That Blocks Ticket Generation

A serious defect in Payment may prevent Ticket Generation and Notification from receiving the required payment status.

In this situation:

- The defect should be classified as high priority.
- Developers should identify the incorrect response or interface data.
- The affected booking scenarios should be temporarily blocked or tested using a controlled mock.
- Payment and Ticket Generation should be retested after the correction.
- End-to-end regression testing should be performed.

The system must never treat an unsuccessful or unverified payment as a completed booking.

5.10 Evaluation of LLM-Generated Integration Plans

An LLM can prepare a draft integration plan using the architecture table. However, the generated plan should not be accepted without review.

The test lead must check:

- Whether all six modules are included.
- Whether every given interface is listed.
- Whether the external gateway is considered.
- Whether the cancellation path is included.
- Whether stubs and drivers are correctly identified.
- Whether payment failure and duplicate callbacks are covered.
- Whether entry and exit criteria are measurable.
- Whether the weekly schedule is realistic.

5.11 Recommended Integration Test Plan

A combined integration plan is recommended for RailEase.

The interface-based plan should be used to verify each module connection. The scenario-based plan should verify complete booking and cancellation workflows. The build-schedule plan should be used to monitor weekly progress and module readiness.

Entry criteria: Unit-tested modules, approved interface specifications, prepared staging environment, and available test data.

Exit criteria: All critical interfaces and business scenarios pass, payment and ticket status remain consistent, and no unresolved critical integration defects remain.

5.12 Conclusion for Task 5

The comparison showed that integration test planning must cover both individual interfaces and complete business workflows. A combined plan provides better traceability, schedule control, and functional coverage. For RailEase, this approach helps ensure that payment, ticket generation, refund processing, and notification work together correctly.

TASK 6 – SYSTEM TESTING: COMPARISON OF SYSTEM TEST FOCUS AREAS

6.1 Introduction

System testing is performed after integration testing to verify the complete application in an environment similar to actual usage. It checks whether the entire system satisfies functional and non-functional requirements.

RailEase must support train search, booking, payment, ticket generation, cancellation, refund processing, and notifications. The system must also meet the specified quality targets:

- Response time of 3 seconds or less at 1,500 concurrent users.
- Availability of 99.5%.
- No double allotment of a berth.
- Payment status and ticket status must always agree.

Three system testing focus areas are compared:

1. Functional and usability testing.
2. Performance, load, and stress testing.
3. Recovery, security, and compatibility testing.

6.2 Functional and Usability System Testing

Functional testing checks whether the complete application performs the expected operations.

For RailEase, important functions include:

- Searching for trains.
- Checking seat availability.
- Booking tickets.
- Applying fare rules.
- Processing payments.
- Generating e-tickets.
- Cancelling tickets.

- Calculating refunds.
- Sending notifications.

Usability testing checks whether booking clerks, travel agents, and passengers can use the system easily.

Advantages:

- Verifies important business operations.
- Helps identify functional defects.
- Checks the user interface.
- Improves user experience.
- Supports acceptance testing.

Limitations:

- Does not measure high-load performance in detail.
- May not expose security weaknesses.
- Depends on the quality of test scenarios.

6.3 Performance, Load, and Stress Testing

Performance testing evaluates how the system behaves under different levels of user activity.

Load testing checks the system under expected user loads, while stress testing examines its behaviour beyond normal operating capacity.

For RailEase, the assignment requires testing at:

- 400 concurrent users.
- 1,500 concurrent users.
- 3,000 concurrent users.

Apache JMeter can be used to simulate multiple users performing train searches and booking operations.

Important measurements include:

- Average response time.
- 90th-percentile response time.
- Error percentage.
- Throughput.
- Resource utilisation.

Advantages:

- Identifies performance bottlenecks.
- Helps estimate system capacity.
- Checks response-time targets.
- Supports preparation for festival-season demand.

Limitations:

- Requires a realistic test environment.
- Load generation needs suitable resources.
- Results may differ from production because of network and data differences.

6.4 Recovery, Security, and Compatibility Testing

This focus area checks whether the application handles failures safely and protects user information.

Recovery testing checks how the application behaves when payment gateways, servers, or network connections fail.

Security testing checks authentication, session handling, access control, and input validation.

Compatibility testing verifies whether the application works across supported browsers and devices.

Advantages:

- Helps identify security weaknesses.
- Checks failure recovery.

- Improves reliability.
- Supports different devices and browsers.

Limitations:

- Requires specialised testing knowledge.
- Security testing may need additional tools.
- Compatibility testing increases the number of environments to maintain.

6.5 System Testing Comparison

Criteria	Functional/Usability	Performance/Load	Recovery/Security/Compatibility
Main focus	Correct operations	Speed and capacity	Safety and reliability
Main tool	Selenium	JMeter	Security and browser tools
Main defects	Incorrect workflow	Slow response	Security and recovery failures
User involvement	High	Limited	Moderate
Critical importance	High	Very high	Very high

6.6 System Test Cases

A. Functional and Usability Test Cases

Test ID	Scenario	Expected result
F1	Search for a valid train	Correct train details displayed
F2	Search for an unavailable train	No availability message displayed
F3	Book one passenger	Booking request accepted
F4	Book six passengers	Booking accepted if seats are available

F5	Book seven passengers	Booking rejected
F6	Apply senior citizen concession	Correct concession applied
F7	Cancel ticket 48 hours before departure	Correct deduction applied
F8	View generated ticket	Correct ticket details displayed

B. Performance Test Cases

Test ID	Concurrent users	Operation	Expected result
P1	400	Train search	Stable response
P2	400	Booking	Successful completion
P3	1,500	Train search	Response time ≤3 seconds
P4	1,500	Booking	Acceptable response and error rate
P5	3,000	Train search	Capacity and degradation measured
P6	3,000	Booking	System stability measured
P7	1,500	Repeated search	No unexpected failures
P8	3,000	Sustained load	Resource and recovery behaviour recorded

C. Recovery, Security, and Compatibility Test Cases

Test ID	Scenario	Expected result
R1	Payment gateway timeout	Transaction remains recoverable
R2	Server restart during booking	Booking state is recovered safely
R3	Duplicate payment callback	No duplicate payment record
R4	Invalid login attempt	Access is denied
R5	Invalid input submission	Input is validated
R6	Expired session	User must authenticate again

R7	Browser compatibility	Application works in supported browsers
R8	Mobile compatibility	Important booking functions work on supported devices

6.7 JMeter Load-Test Design

Apache JMeter can be used to create a load test for the Search and Booking workflow.

Test flow:

Start Test

|

v

Create Virtual Users

|

v

Open RailEase

|

v

Search for Train

|

v

Check Availability

|

v

Submit Booking

|

v

Record Response Time

|

v

Calculate Average and 90th Percentile

|

v

Generate Test Report

JMeter Configuration

Parameter	Value
Tool	Apache JMeter
Test operation	Search and booking
User groups	400, 1,500, 3,000
Main measurement	Response time
Additional measurements	Throughput and error rate
Acceptance target	≤3 seconds at 1,500 users

The test should use realistic search data and avoid repeatedly sending identical requests if the objective is to represent normal passenger activity.

6.8 Response-Time Calculations

Average response time is calculated using:

Average Response Time = Total Response Time / Number of Requests

The 90th percentile represents the response time within which 90% of the requests are completed.

For example, if the average response time is 2.5 seconds and the 90th-percentile response time is 2.9 seconds, the system is within the 3-second target for those measurements.

The actual response-time requirement should be checked using the metric agreed with the railway. If the requirement applies to average response time, the average must be checked. If it applies to individual or percentile response time, that metric must also be evaluated.

Illustrative Performance Results

The following values are sample results for explaining the comparison. They are not actual JMeter measurements.

Concurrent users	Average response time	90th percentile	Interpretation
400	1.2 sec	1.8 sec	Acceptable sample result

1,500	2.4 sec	2.9 sec	Within 3-second sample target
3,000	4.8 sec	6.2 sec	Performance target exceeded

These values show how response time may increase as the number of concurrent users increases.

6.9 Trial-Week Problems and Testing Focus

The trial week identified four major problems:

1. A waitlisted ticket was confirmed twice for the same berth.
2. Refund was calculated using the wrong cancellation slab.
3. Payment succeeded but no ticket was generated.
4. The website slowed down when 400 users searched together.

Trial problem	**Functional testing**	**Performance testing**	**Recovery/security testing**
Double berth allotment	Strong detection	Limited detection	Recovery/concurrency checks may expose it
Incorrect refund slab	Strong detection	Not primary focus	Not primary focus
Payment without ticket	End-to-end scenario	Limited detection	Strong recovery detection
Slow train search	Limited under normal load	Strong detection	Limited detection

No single testing focus area identifies every problem. Therefore, all three areas are required.

6.10 Staging and Production Environment Differences

Testing results may differ between staging and production because of:

- Different numbers of users.
- Different database sizes.

- Different server capacity.
- Network latency.
- Payment gateway response times.
- Different security configurations.
- Different caching and load-balancing settings.

The 18 months of anonymised booking data provided in the assignment is useful for realistic testing. However, it may not fully represent festival-season booking patterns.

The team should therefore generate additional synthetic peak-load data and compare the results with the historical booking patterns.

6.11 Recommended Five-Week Effort Allocation

The following is a proposed allocation of the five-week system-testing effort.

Testing focus	**Proposed effort**	**Reason**
Functional and usability	35%	Critical booking and refund operations
Performance and load	40%	Festival-season user demand
Recovery, security and compatibility	25%	Payment reliability and user protection
Total	**100%**	

Performance receives the largest share because the system must support 1,500 concurrent users within the specified response-time target. Functional testing remains equally important for preventing incorrect bookings, payments, and refunds.

6.12 Why Load Testing Should Not Be Run Only Once

Running a load test only one week before release is risky because performance problems may be discovered too late.

A better schedule is:

- Early performance testing after the first integrated build.
- Intermediate load testing after major defects are corrected.
- Full load testing before final release.
- A smaller regression performance test after important changes.

This allows developers to identify bottlenecks early and verify whether corrections improve performance.

6.13 Recommended System Testing Approach

A balanced system-testing approach is recommended. Functional testing should verify the business operations, performance testing should validate the festival-season capacity, and recovery/security/compatibility testing should verify safe operation.

The three approaches should not be treated as alternatives where only one can be used. They address different quality risks and must be combined.

6.14 Conclusion for Task 6

System testing helps verify whether RailEase works correctly as a complete application. Functional, performance, recovery, security, and compatibility testing each provide important information about software quality. A balanced testing approach is necessary to meet the railway's reliability, availability, and response-time targets before the January festival season.

TASK 7 – REGRESSION TESTING: COMPARISON OF TEST SELECTION STRATEGIES

7.1 Introduction

Regression testing is performed after software changes to ensure that previously working functionality has not been affected.

RailEase developers may frequently change fare rules, refund slabs, booking conditions, and payment handling. Such changes can introduce new defects into existing features.

The assignment provides 650 existing manual test cases. Running all 650 cases takes the three testers four days. Therefore, selecting the most important regression tests is necessary to reduce testing time without losing important defect coverage.

Three strategies are compared:

1. Retest-all.
2. Impact-based selection.
3. Prioritised suite using fault history.

7.2 Comparison of Regression Strategies

A. Retest-All

Retest-all executes every existing test case after each software change.

For RailEase, all 650 test cases are executed even when only the refund calculation function is modified.

Advantages:

- Provides broad regression coverage.
- Does not depend on change-impact analysis.
- Reduces the chance of missing defects in unrelated modules.
- Useful before major releases.

Limitations:

- Requires more time.

- Repeats many tests that may be unrelated to the change.
- Increases manual effort.
- May delay weekly releases.

B. Impact-Based Test Selection

Impact-based testing selects test cases linked to the changed module and its dependent modules.

For example, if the refund slab changes, the test lead selects refund, cancellation, payment reconciliation, and notification-related cases.

Advantages:

- Reduces the number of tests.
- Focuses on affected modules.
- Uses the traceability sheet.
- Saves execution time.

Limitations:

- Depends on accurate traceability information.
- Unrecorded dependencies may be missed.
- A change may affect a module that is not directly linked.

C. Prioritised Suite Using Fault History

This strategy ranks test cases based on:

- Previous fault detection.
- Business importance.
- Coverage of changed code.
- Defect severity.
- Failure history.

High-priority tests are executed first.

Advantages:

- Important tests run early.
- Helps identify critical defects quickly.
- Useful when testing time is limited.
- Supports risk-based regression testing.

Limitations:

- Requires historical test information.
- Old fault patterns may not predict new defects.
- Lower-priority tests may not run before release.

7.3 Comparison Table

Criteria	Retest-All	Impact-Based	Prioritised Suite
Test selection	All tests	Related tests	Ranked tests
Execution effort	High	Moderate	Flexible
Fault history	Not required	Not essential	Important
Traceability	Helpful	Essential	Helpful
Risk of missed faults	Lower if suite is complete	Depends on impact accuracy	Depends on ranking
Best use	Major releases	Regular changes	Time-limited releases

7.4 Representative Regression Test Suite

For practical comparison, a representative suite of 60 tests can be used instead of the full 650-case suite.

The 60 tests may be divided as follows:

Module	Number of tests
Train Search and Availability	10
Booking	12
Payment	10
Ticket Generation	8
Cancellation and Refund	12
Notification	8
Total	**60**

Assume the change is made to the refund slab from 12 hours to 10 hours.

The impact-based suite should include cancellation and refund cases, payment refund handling, and relevant notification cases.

7.5 Three Seeded Regression Faults

Three faults are introduced into the changed version.

Fault ID	Regression fault	Expected behaviour
RF1	Incorrect refund boundary	Updated 10-hour boundary must be followed
RF2	Refund amount calculated using wrong fare	Correct fare must be used
RF3	Refund success not updated in notification	Correct refund status must be communicated

7.6 Illustrative Regression Test Selection

The following table uses an example 60-test suite. The numbers are illustrative and must be replaced with actual test execution results.

Strategy	Tests selected	Faults detected	Execution time
Retest-all	60	3	60 minutes
Impact-based	24	2	24 minutes
Prioritised	30	3	30 minutes

For this example, each test is assumed to require approximately one minute.

7.7 Fault Detection Percentage

Fault Detection Percentage is calculated as:

Fault Detection Percentage = (Faults detected / Total seeded faults) × 100

There are three seeded faults.

Retest-all:

= (3 / 3) × 100

= 100%

Impact-based:

= (2 / 3) × 100

= 66.67%

Prioritised:

= (3 / 3) × 100

= 100%

These are illustrative calculations based on the example results.

7.8 Time Saved Against Retest-All

Time Saved = Retest-All Time − Selected Strategy Time

Percentage Time Saved = (Time Saved / Retest-All Time) × 100

Retest-all:

60 − 60 = 0 minutes

Time saved = 0%

Impact-based:

60 − 24 = 36 minutes

Percentage saved = (36 / 60) × 100

= 60%

Prioritised:

60 − 30 = 30 minutes

Percentage saved = (30 / 60) × 100

= 50%

Comparison Table

Strategy	Fault detection	Time saved	Percentage saved
Retest-all	100%	0 min	0%
Impact-based	66.67%	36 min	60%
Prioritised	100%	30 min	50%

7.9 Average Percentage of Faults Detected (APFD)

APFD measures how quickly a test suite detects faults based on the order in which tests are executed.

The formula is:

APFD = 1 − (TF1 + TF2 + ... + TFn) / (n × m) + 1 / (2n)

Where:

- n = Number of test cases.
- m = Number of faults.
- TFi = Position of the first test case that detects fault i.

Assume that three faults are detected at test positions 2, 5, and 8 in a prioritised suite containing 30 tests.

Therefore:

$n = 30$

$m = 3$

$TF1 = 2, TF2 = 5, TF3 = 8$

$APFD = 1 - (2 + 5 + 8) / (30 \times 3) + 1 / (2 \times 30)$

$APFD = 1 - 15/90 + 1/60$

$APFD = 1 - 0.1667 + 0.0167$

APFD = 0.85 or 85%

This is an illustrative calculation. Actual APFD must use the first detection position of each seeded fault in the executed test order.

7.10 Risk of an Outdated Traceability Sheet

Impact-based selection depends on the traceability sheet to identify affected tests.

Suppose a refund calculation change also affects the Notification module, but the traceability sheet does not show this relationship. The impact-based strategy may select only refund tests and miss the notification defect.

To reduce this risk:

- Update traceability after every requirement change.
- Review dependent modules.
- Include critical end-to-end tests.
- Run a smoke suite after each build.
- Review changes with developers and testers.

7.11 Test-Suite Minimisation and Obsolete Tests

Test-suite minimisation reduces repeated or unnecessary test cases while preserving important coverage.

Obsolete tests should not be deleted immediately. They should first be reviewed to determine whether the requirement has genuinely been removed or replaced.

Tests that always pass should also be examined. They may be valid regression tests, or they may contain weak assertions that fail to detect incorrect behaviour.

The team should:

- Review test purpose.
- Check whether assertions are meaningful.
- Identify duplicate tests.
- Retain important historical regression cases.
- Remove only tests that are confirmed to be obsolete.

7.12 Recommended Regression Strategy

A prioritised regression suite is recommended for weekly releases because it gives early attention to high-risk and business-critical functionality.

The strategy should be supported by impact analysis and a small mandatory smoke suite. For major releases, a wider regression suite or complete retest should be performed.

Recommended Smoke Suite

A proposed smoke suite contains approximately 15 tests:

- 2 train search tests.
- 3 booking tests.
- 3 payment tests.
- 2 ticket generation tests.
- 3 cancellation and refund tests.
- 2 notification tests.

These tests verify whether the main application functions are working after a new build.

7.13 Evaluation of LLM-Based Regression Test Selection

An LLM can suggest regression tests based on a code commit message. However, a commit message may not contain all affected modules or dependencies.

For example, a message saying "Update refund calculation" may not mention the payment reconciliation or notification changes caused by the same update.

Therefore, an LLM should be given additional information such as:

- Changed source files.
- Requirement traceability.
- Module dependencies.
- Previous defect history.
- Test coverage information.

The suggested tests must be reviewed by the test lead before execution. AI-based selection should support human decision-making rather than replace it.

7.14 Conclusion for Task 7

The comparison demonstrated that retest-all provides broad coverage but requires more time. Impact-based testing reduces execution effort but depends on accurate traceability. Prioritised regression testing provides a practical balance by executing high-risk tests first. For RailEase, a prioritised suite supported by impact analysis and a mandatory smoke suite is recommended for weekly releases.

TASK 8 – ALPHA, BETA AND ACCEPTANCE TESTING

8.1 Introduction

Acceptance testing is performed to determine whether a software system is ready for use by its intended users. It checks whether the application satisfies business requirements and provides the expected user experience.

RailEase will be used by booking clerks, travel agents, passengers, and railway commercial officers. Therefore, testing must involve different types of users before the system is released.

The assignment considers three approaches:

1. Alpha testing.
2. Beta testing.
3. Formal acceptance testing.

8.2 Alpha Testing

Alpha testing is performed by a small group of internal users in a controlled environment. Developers and testers are available to observe the users and identify problems.

For RailEase, the alpha group contains:

- 4 booking clerks.
- 3 travel agents.

The users will test the system in a controlled staging environment.

Objectives

- Verify the booking workflow.
- Check the usability of the interface.
- Identify missing features.
- Verify cancellation and refund operations.
- Collect feedback from experienced users.

Alpha Test Plan

Item	Description
Test participants	4 booking clerks and 3 travel agents
Environment	Controlled staging environment
Duration	Proposed 3–4 days
Test activities	Search, booking, payment, cancellation
Observation	Testers observe user activity
Defect reporting	Defect tracker and feedback forms
Expected result	Functional and usability issues identified

Alpha Test Scenarios

1. Search for a train using source and destination.
2. Check available seats.
3. Book a Sleeper ticket.
4. Book an AC ticket.
5. Apply senior citizen concession.
6. Make a payment.
7. View the generated e-ticket.
8. Cancel a ticket.
9. Verify the refund amount.
10. Check booking and cancellation notifications.

8.3 Beta Testing

Beta testing allows selected external users to use the application in a more realistic environment.

For RailEase, 50 regular travellers will test the staging system using their own devices.

Objectives

- Collect feedback from real users.
- Identify usability problems.
- Check device compatibility.
- Understand actual user behaviour.
- Identify defects that internal testers may miss.

Beta Feedback Form

RailEase Beta Testing Feedback Form

1. Were you able to log in successfully?
2. Was the train search process easy to understand?
3. Were train details displayed correctly?
4. Was seat availability easy to check?
5. Were fare details clearly displayed?
6. Was the booking process easy to complete?
7. Did payment work as expected?
8. Was the e-ticket generated successfully?
9. Was the cancellation process understandable?
10. Was the refund amount displayed correctly?

Feedback rating:

- 5 – Excellent.
- 4 – Good.
- 3 – Average.
- 2 – Poor.
- 1 – Very poor.

8.4 Formal Acceptance Testing

Formal acceptance testing is conducted by the authorised business representative. For RailEase, the commercial officer reviews the system against approved acceptance criteria.

The objective is to confirm that the application satisfies business, operational, and legal requirements before release.

Responsibilities of the Commercial Officer

- Review business requirements.
- Verify fare and refund rules.
- Review booking and cancellation workflows.
- Check payment and ticket consistency.
- Review unresolved defects.
- Approve or reject the final release.

8.5 Ten Acceptance Test Cases

Test ID	Acceptance scenario	Measurable acceptance criterion
AT1	Train search	Valid searches return correct results
AT2	Seat availability	A berth is never assigned to more than one passenger
AT3	Passenger booking	1–6 passengers can be booked successfully when seats are available
AT4	Tatkal booking	30% surcharge is correctly applied
AT5	Senior citizen concession	Eligible passengers receive the 40% base fare concession
AT6	AC ticket GST	5% GST is applied to AC classes only
AT7	Payment	Payment and ticket status always agree
AT8	Ticket generation	Successful payment leads to a valid e-ticket

Additional Measurable Acceptance Criteria

The following are proposed operational criteria that should be approved by the railway:

- At least 95% of successful booking attempts should complete within three minutes under the agreed UAT conditions.
- The system must meet the 3-second response-time target at 1,500 concurrent users.
- No duplicate berth allocation should occur in the acceptance test scenarios.
- Payment and ticket status must remain consistent.
- Critical booking and refund scenarios must pass.
- No unresolved critical defects should remain at final sign-off.

8.6 Simulated User Evaluation

The assignment requests testing with at least five volunteers. The following is an example of how the issue classification can be recorded. It is not actual volunteer feedback.

Approach	Example issue	Issue type	Severity
Alpha	Fare details are difficult to locate	Usability	Medium
Alpha	Refund slab is displayed incorrectly	Defect	High
Beta	Booking page does not display correctly on one device	Compatibility	Medium
Beta	Search results take too long to load	Performance defect	High
Acceptance	Refund calculation does not follow approved policy	Business defect	Critical

Issue Types

Defect: The system does not behave according to the requirement.

Usability issue: The system works, but users find it difficult to understand or operate.

Change request: The user requests an additional feature or modification that is not part of the current approved requirement.

8.7 Comparison of Alpha, Beta and Acceptance Testing

Criteria	Alpha	Beta	Formal Acceptance
Participants	Internal users	External travellers	Commercial officer
Environment	Controlled	Realistic staging	Approved UAT
Main focus	Functionality and usability	Real-user experience	Business approval
Feedback	Direct observation	Feedback forms	Formal review
Main limitation	Tester bias	Inconsistent feedback	Depends on acceptance criteria

8.8 Handling Unresolved Beta Defects

Not every defect found during beta testing must automatically block release. Defects should be classified according to severity and business impact.

Critical defects: These must be fixed before release. Examples include double berth allocation, incorrect payment status, and serious refund calculation errors.

High-priority defects: These should normally be fixed before release, especially if they affect important booking activities.

Medium-priority defects: These may be accepted temporarily only with documented approval, a suitable workaround, and a planned correction date.

Low-priority issues: Minor usability improvements may be scheduled for a later release if they do not violate acceptance criteria.

The commercial officer must approve any release decision involving unresolved defects.

8.9 Recommended Acceptance Sequence

The following sequence is proposed within the five-week testing period.

Week	Activity	Participants
Week 3	Alpha testing	Booking clerks and travel agents
Week 4	Beta testing	50 regular travellers
Week 5	Formal acceptance testing	Commercial officer

This sequence may be adjusted according to module readiness and defect correction.

Alpha testing should occur first because it provides controlled feedback. Beta testing follows after major defects are corrected. Formal acceptance testing is performed near the end of the release process.

8.10 Can 50 Beta Users Represent 1.5 Lakh Daily Travellers?

Fifty beta users can provide useful information about usability, navigation, and common functional issues. However, they cannot fully represent 1.5 lakh daily travellers.

The beta group may not cover:

- All age groups.
- All geographical regions.
- All device types.
- All network conditions.
- All booking behaviours.
- Festival-season traffic patterns.

Therefore, beta testing should be supported by automated performance testing, load testing, and additional representative test data.

The results from 50 users should be treated as limited user feedback rather than proof that the application can support 1.5 lakh daily travellers.

8.12 Conclusion for Task 8

The comparison showed that acceptance testing is important because software must satisfy the expectations of its actual users. Alpha testing, beta testing, and formal acceptance testing provide different types of feedback. For RailEase, using all three in a planned sequence provides better confidence before the system is released for the January festival travel season.

TASK 9 – INTEGRATED MULTI-LEVEL TESTING: COMPARISON OF COMPLETE TEST STRATEGIES

9.1 Introduction

Integrated multi-level testing combines different testing levels into one complete strategy. It includes unit testing, integration testing, system testing, regression testing, alpha testing, beta testing, and formal acceptance testing.

For RailEase, all these levels are important because the system handles financial transactions, passenger information, seat allocation, and large numbers of booking requests.

The assignment compares three complete strategies:

1. Bottom-heavy strategy.
2. Top-heavy strategy.
3. Balanced testing pyramid strategy.

An additional AI-assisted strategy is also evaluated.

9.2 Comparison of Complete Testing Strategies

A. Bottom-Heavy Strategy

A bottom-heavy strategy gives more effort to unit and integration testing. It performs fewer system tests and shorter acceptance testing.

Advantages:

- Finds defects early.
- Provides detailed unit coverage.
- Helps isolate integration problems.
- Reduces the cost of fixing some defects.

Limitations:

- May provide insufficient real-user testing.
- Large-scale performance problems may remain undetected.
- Complete business workflows may receive less attention.

B. Top-Heavy Strategy

A top-heavy strategy spends less effort on unit testing and gives more attention to system testing and acceptance testing.

Advantages:

- Focuses on complete user workflows.
- Tests realistic application behaviour.
- Provides direct user feedback.
- Helps identify end-to-end issues.

Limitations:

- Defects may be discovered late.
- Debugging may be difficult.
- System testing requires more time and resources.
- Repeated testing can become expensive.

C. Balanced Testing Pyramid Strategy

A balanced strategy uses many automated unit tests, a moderate number of integration tests, and focused system and acceptance testing.

Advantages:

- Provides coverage at different levels.
- Finds calculation defects early.
- Supports automated regression testing.
- Provides end-to-end verification.
- Balances cost and testing effort.

Limitations:

- Requires proper test automation.
- Needs coordination between testers and developers.

- Test environments must be maintained.
- Requires clear test prioritisation.

9.3 Comparison Table

Criteria	Bottom-Heavy	Top-Heavy	Balanced Pyramid
Unit testing	High	Low	High
Integration testing	High	Low	Moderate
System testing	Low	High	Focused
Regression testing	Strong at lower levels	Mostly end-to-end	Automated and risk-based
Acceptance testing	Limited	High	Staged
Early defect detection	High	Low	High
User-level confidence	Moderate	High	High

9.4 Proposed Multi-Level Testing Structure

The recommended testing structure is:

UNIT TESTING

|

v

INTEGRATION TESTING

|

v

SYSTEM TESTING

|

v

REGRESSION TESTING

|

v

ALPHA TESTING

|

v

BETA TESTING

|

v

FORMAL ACCEPTANCE TESTING

|

v

FINAL RELEASE

Each level has separate entry and exit criteria.

9.5 Entry and Exit Criteria for Each Level

Testing level	**Entry criteria**	**Exit criteria**
Unit	Functions are available	Critical unit tests pass
Integration	Unit tests pass	Critical interfaces work
System	Integrated build is ready	Functional and quality targets verified
Regression	A corrected build is available	Selected regression tests pass
Alpha	Stable internal test build	Important internal issues resolved
Beta	Major defects corrected	User feedback reviewed
Acceptance	Approved UAT build	Commercial officer signs off

9.6 Proposed Five-Week Effort Allocation

The following is a proposed allocation of the total testing effort. It is a planning estimate and should be adjusted using actual team capacity.

Testing level	**Percentage of effort**
Unit testing	20%
Integration testing	20%

System testing	25%
Regression testing	15%
Alpha testing	7%
Beta testing	8%
Formal acceptance testing	5%
Total	**100%**

System testing receives the largest share because RailEase must meet important performance and reliability targets before the festival season.

9.7 Ten Seeded Faults for Prototype Evaluation

The prototype should contain ten intentionally introduced faults.

Fault ID	Fault description	Testing level expected to detect it
F1	GST charged on Sleeper	Unit
F2	Incorrect senior citizen age boundary	Unit
F3	Incorrect 48-hour refund boundary	Unit
F4	Refund deduction applied once per ticket	Unit
F5	Wrong passenger count passed to Payment	Integration
F6	Payment amount sent in paise	Integration
F7	Ticket not generated after successful payment	Integration/System
F8	Duplicate berth allocation	System/Integration
F9	Notification receives incorrect date format	Integration/System
F10	Search becomes slow under high load	Performance/System

These faults represent different types of problems that may occur at different testing levels.

9.8 Illustrative Strategy Evaluation

The following table demonstrates a possible comparison of the three strategies. The values are illustrative and must be replaced with actual prototype execution results.

Measurement	Bottom-Heavy	Top-Heavy	Balanced Pyramid
Unit faults detected	4	2	4
Integration faults detected	3	2	3
System-level faults detected	1	3	3
Total faults detected	8	7	10
Main testing effort	Unit/integration	System/acceptance	All levels
Expected defect leakage	Moderate	Higher for low-level faults	Lower
Regression effort	Moderate	High	Lower with automation

The table is an example of expected outcomes, not a measured experiment.

9.9 Defect Removal Efficiency

Defect Removal Efficiency (DRE) measures the percentage of identified defects removed before release.

DRE = (Defects removed before release / Total defects identified) × 100

For example, if 18 defects are identified and 16 are corrected before release:

DRE = (16 / 18) × 100

DRE = 88.89%

This measurement can be calculated separately for each testing strategy after actual defect records are available.

9.10 Cost per Defect

Cost per Defect = Total Testing Cost / Number of Defects Detected

For example, if a testing strategy requires ₹12,000 and detects 10 defects:

Cost per Defect = ₹12,000 / 10

= ₹1,200 per defect.

This is an illustrative calculation. Actual cost should include tester effort, tool costs, environment costs, and defect investigation effort.

9.11 Expected Defect Leakage

Defect leakage refers to defects that are not identified during testing but are discovered after release.

A bottom-heavy strategy may identify calculation and interface errors early but can miss user-level and performance problems if system testing is too limited.

A top-heavy strategy may identify end-to-end problems but can miss internal defects until late in the process.

A balanced strategy provides better coverage across all levels and is expected to reduce defect leakage when test cases are properly designed.

9.12 Design Decision Table

Decision	**Justification (Evidence)**	**What If Not?**
Select balanced testing pyramid	Covers all testing levels	Important defects may remain undetected
Automate unit tests	Supports fast execution	Repeated manual testing increases effort
Use mock payment gateway	Supports isolated testing	External dependency may delay testing
Use thread-based integration	Verifies booking and cancellation workflows	Interface defects may be found late

Perform load testing	Verifies 1,500-user target	Festival traffic may affect performance
Use prioritised regression	Reduces weekly testing effort	Full retest may delay releases
Conduct alpha and beta testing	Collects user feedback	Usability issues may remain
Require formal acceptance	Confirms business approval	Release may not satisfy railway needs

9.13 Comparison with Other Systems

The balance of testing levels changes according to the type of application.

Testing level	**RailEase Railway System**	**Airline Seat Allocation**	**ATM Network Switch**	**School Attendance App**
Unit	High	High	High	Moderate
Integration	High	Very high	Very high	Moderate
System	Very high	Very high	Very high	Moderate
Performance	Very high	Very high	Very high	Moderate
Security	High	Very high	Very high	High
Regression	High	Very high	Very high	High
Acceptance	Very high	High	High	Very high

Airline Seat-Allocation System

An airline seat-allocation system has similar risks to RailEase because the same seat must not be assigned to multiple passengers. Therefore, concurrency testing, integration testing, and performance testing receive high priority.

ATM Network Switch

An ATM network switch handles financial transactions between banks and ATM terminals. Transaction accuracy, security, availability, and recovery testing are extremely important. Integration and security testing receive a larger share of effort.

School Attendance Application

A school attendance application generally has lower transaction complexity than a railway reservation system. Functional testing, usability testing, data accuracy, and access control are important. Performance testing is still required during peak attendance periods.

9.14 Limitations of Historical Anonymised Booking Data

The assignment provides 18 months of anonymised booking data. This data is useful for preparing realistic test cases, but it may not fully represent festival-season traffic.

Historical data may not include:

- Sudden increases in Tatkal booking requests.
- New travel patterns.
- Large numbers of users searching at exactly the same time.
- Changes in passenger behaviour.
- New payment methods.
- Unexpected network conditions.

One possible difference between expectation and actual testing is that a system may perform well with 400 users but experience a significant increase in response time at 1,500 users. This can happen because of database contention, limited server resources, or external service delays.

9.15 Conclusion for Task 9

The comparison of complete testing strategies showed that no single testing level is sufficient to ensure application quality. A balanced strategy provides a suitable combination of early defect detection, interface verification, system validation, regression coverage, and user acceptance. For RailEase, this approach provides better confidence in the application's readiness for festival-season operation.

TASK 10 – RE-ANALYSE AND ADAPT THE MULTI-LEVEL TEST STRATEGY

10.1 Introduction

Software systems frequently change as new requirements are introduced. A testing strategy that was suitable for an earlier version may not be sufficient for a newer version.

The RailEase system introduces several major changes:

- A mobile application.
- UPI payments.
- Wallet payments.
- A separate seat-recommendation microservice.
- Tatkal booking at 10:00 AM.
- Up to 15,000 simultaneous users.
- Fortnightly software releases.
- Reduction of the testing team from three testers to two.
- Requirement for traceable evidence that every critical test was executed before each release.

These changes increase system complexity and create additional testing responsibilities. Therefore, the original strategy from Task 9 must be reviewed and adapted.

10.2 Comparison of Adaptation Strategies

A. Retain and Prioritise

This strategy keeps the original balanced testing strategy but selects only high-risk test cases for every release.

Advantages:

- Requires fewer changes to the existing process.
- Uses existing test cases.
- Reduces testing effort.

- Can be implemented quickly.

Limitations:

- Does not fully address the new microservice architecture.
- Manual testing may become difficult with two testers.
- High-load requirements may exceed the original test capacity.
- Shortening beta testing may reduce user feedback.

B. Modify with an Automated Multi-Level Pipeline

This strategy introduces automated unit, integration, and regression tests into a continuous integration pipeline. Manual system and acceptance testing are retained for new features and business validation.

Advantages:

- Reduces repeated manual effort.
- Provides faster feedback.
- Supports fortnightly releases.
- Maintains repeatable test execution.
- Produces automated test reports.

Limitations:

- Requires initial automation effort.
- Automated scripts need maintenance.
- Environment configuration must be managed.
- Manual review is still necessary.

C. Redesign with Contract Tests and Staged Rollout

This strategy introduces consumer-driven contract testing between services, automated load tests for every release, and staged deployment instead of depending on a long beta period.

Advantages:

- Verifies service communication.
- Helps identify API compatibility problems.
- Supports microservice testing.
- Reduces release risk through staged deployment.
- Provides repeated load-test evidence.

Limitations:

- Requires more advanced test infrastructure.
- Contract tests need careful maintenance.
- Canary deployment requires monitoring and rollback mechanisms.
- Initial implementation effort is high.

10.3 Comparison Table

Criteria	**Retain + Prioritise**	**Automated Pipeline**	**Contract Tests + Staged Rollout**
Initial effort	Low	Moderate to high	High
Automation	Limited	High	High
Microservice support	Limited	Good	Very good
Regression testing	Selected manual tests	Automated	Automated
Load testing	Occasional	Scheduled	Every release
Release readiness	Moderate	High	High
Maintenance	Moderate	Moderate	High initially

10.4 Impact Analysis of Changed Requirements

The new requirements affect several assumptions made in Task 9.

Original assumption	New requirement	Impact
Six modules	Separate seat-recommendation microservice	More interfaces and dependencies
Three testers	Two testers	Reduced manual testing capacity
Five-week release window	Fortnightly releases	Faster testing cycle required
1,500 concurrent users	15,000 simultaneous users	Much higher performance demand
Web application	Web and mobile application	Additional compatibility testing
External payment gateway	UPI and wallet payments	More payment scenarios
Existing test evidence	Every critical test must be traceable	Stronger reporting and audit requirements

The old strategy cannot be used unchanged because it was designed for a smaller user load, a larger testing team, and a longer release period.

10.5 Impact of Using the Old Strategy Unchanged

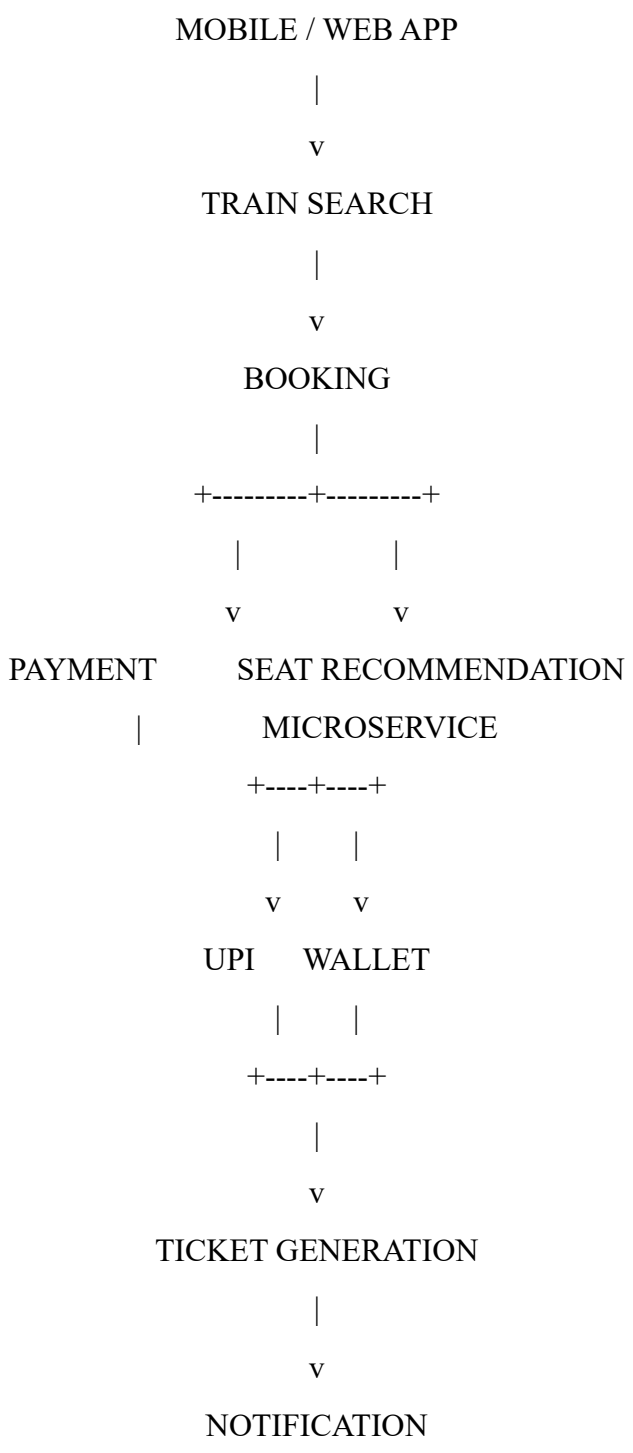
If the old strategy is continued without modification, several problems may occur:

- Manual regression testing may not finish within two weeks.
- Mobile application defects may not be identified.
- UPI and wallet payment conditions may remain untested.
- Seat-recommendation service failures may affect booking.
- Performance problems may appear at 15,000 users.
- Critical test execution evidence may be incomplete.
- Release delays may increase.

Therefore, automation, test prioritisation, and service-level testing are necessary.

10.6 Updated Module Structure

The updated RailEase architecture contains the original modules along with the mobile application, new payment methods, and the seat-recommendation microservice.



CANCELLATION

|

v

PAYMENT

|

v

REFUND PROCESSING

This is a logical overview. The exact service connections must follow the final approved architecture.

10.7 Updated Testing Requirements

Mobile Application Testing

The mobile application must be tested for:

- Login and authentication.
- Train search.
- Seat availability.
- Booking.
- Payment.
- Ticket display.
- Cancellation.
- Notification delivery.
- Different screen sizes.
- Supported operating systems.

UPI Payment Testing

UPI testing should include:

- Successful payment.
- Failed payment.

- Pending payment.
- Payment timeout.
- Duplicate callback.
- Incorrect transaction reference.
- Payment completed after session expiry.

Wallet Payment Testing

Wallet testing should include:

- Sufficient balance.
- Insufficient balance.
- Successful debit.
- Failed debit.
- Duplicate debit request.
- Wallet balance update.
- Refund to wallet.

Seat-Recommendation Microservice Testing

The separate service should be tested for:

- Valid recommendation response.
- No available recommendation.
- Service timeout.
- Invalid response.
- Incorrect seat details.
- Service unavailable.
- Compatibility with the Booking module.

10.8 Load Testing for 15,000 Simultaneous Users

The new system must support up to 15,000 simultaneous users during Tatkal booking at 10:00 AM.

The original performance target of 1,500 concurrent users is no longer sufficient.

The testing team should perform load tests at gradually increasing user levels.

Load level	Concurrent users	Purpose
Level 1	1,500	Baseline comparison
Level 2	5,000	Intermediate load
Level 3	10,000	High-load verification
Level 4	15,000	Required peak load
Level 5	Above 15,000	Stress and capacity evaluation

The system should be monitored for:

- Average response time.
- 90th-percentile response time.
- Error rate.
- Throughput.
- Database performance.
- CPU and memory usage.
- Payment service response.
- Seat availability consistency.

The 3-second response-time target should be checked at the newly required load if the railway confirms that the same target applies at 15,000 concurrent users.

A test at 1,500 users alone cannot prove that the system can support 15,000 users.

10.9 Recalculating Testing Effort for Two Testers

The original assignment provides three testers for five weeks.

Assuming five working days per week and eight working hours per day:

Original capacity:

3 × 5 × 8 = 120 tester-hours per week.

For two testers:

2 × 5 × 8 = 80 tester-hours per week.

For a two-week release cycle:

2 × 10 × 8 = 160 tester-hours.

Therefore, the team has approximately 160 tester-hours per fortnight before meetings, administration, and other non-testing activities.

The available testing effort is lower than before because one tester has been transferred and releases occur more frequently.

Automation can reduce repeated execution effort and allow the two testers to focus on high-risk workflows, defect investigation, and acceptance testing.

10.10 Recommended Automated Testing Pipeline

A continuous integration pipeline can be used to execute tests whenever developers submit changes.

CODE COMMIT

|

v

BUILD SOFTWARE

|

v

UNIT TESTS

|

v

CONTRACT TESTS

|

v

INTEGRATION TESTS

|

v

AUTOMATED REGRESSION

|

v

LOAD TESTS

|

v

MANUAL SYSTEM TESTS

|

v

ACCEPTANCE REVIEW

|

v

RELEASE APPROVAL

|

v

STAGED ROLLOUT

Pipeline Activities

1. Build: Compile and prepare the latest software version.

2. Unit testing: Verify fare, refund, booking, and payment functions.

3. Contract testing: Check whether the microservice interfaces follow the agreed request and response formats.

4. Integration testing: Verify communication between Booking, Payment, Ticket Generation, Notification, and the recommendation service.

5. Regression testing: Execute selected critical and changed-function tests.

6. Load testing: Verify the required user capacity.

7. Manual system testing: Check important user workflows on web and mobile.

8. Acceptance review: Verify business requirements and critical test evidence.

9. Staged rollout: Release to a limited group first, monitor the system, and expand deployment only when the release is stable.

10.11 Critical Test Evidence and Traceability

The railway board requires evidence that every critical test was executed before each release.

Therefore, every critical test must have:

- Unique test case ID.
- Requirement ID.
- Module name.
- Test input.
- Expected result.
- Actual result.
- Execution date.
- Software build number.
- Pass/fail status.
- Tester or automation identity.
- Defect reference, if failed.
- Retest result after correction.

Example Test Evidence Table

Test ID	Requirement	Build	Result	Evidence
CR01	No duplicate berth	Build 1.0	Pass	Execution report
CR02	Correct payment status	Build 1.0	Pass	Payment log
CR03	Ticket after successful payment	Build 1.0	Pass	Ticket record
CR04	Correct refund slab	Build 1.0	Pass	Refund report
CR05	15,000-user load test	Build 1.0	Pending	JMeter report

This table is a format example. Actual release evidence must contain the real execution results.

10.12 Comparison of Three Adaptation Strategies

The following table gives an illustrative evaluation. Actual faults found, effort, and release delay must be measured using the extended prototype.

Criteria	**Retain + Prioritise**	**Automated Pipeline**	**Contract Tests + Staged Rollout**
Fault detection	Depends on selected tests	Strong across repeated tests	Strong at service boundaries
Manual effort	Moderate to high	Lower after setup	Moderate
Release delay	May increase	Reduced after automation	Depends on deployment readiness
Load readiness	Needs additional testing	Scheduled load tests	Repeated load and rollout monitoring
Audit evidence	Manual records	Automated reports	Automated reports and deployment records
Microservice validation	Limited	Good	Strong

10.13 Final Decision: MODIFY

The recommended decision is **MODIFY the existing balanced testing strategy by introducing an automated multi-level pipeline, supported by contract testing and staged rollout**.

The original balanced strategy is still useful because it includes unit, integration, system, regression, and acceptance testing. However, the new requirements make the earlier manual process insufficient.

The modified approach should include:

- Automated unit testing.
- Automated integration and regression testing.

- Contract testing for the seat-recommendation microservice.
- UPI and wallet payment tests.
- Mobile application compatibility testing.
- Repeated performance testing up to the required user load.
- Mandatory critical test evidence.
- Manual system and acceptance testing.
- Staged rollout with monitoring and rollback support.

This approach makes better use of the reduced two-person testing team and supports fortnightly releases.

10.14 Condition Under Which the Decision May Change

The decision may need to change if the railway introduces additional microservices, significantly increases the number of users, or requires a stricter release approval process that cannot be supported by the current two-person testing team.

In such a situation, a more complete redesign with additional automation infrastructure, dedicated performance testing resources, and expanded monitoring may become necessary.

10.16 Risks of AI-Generated Test Selection

AI-generated test selection can help identify test cases affected by a code change. It may analyse changed files, previous defects, and requirement information to suggest useful regression tests.

However, there are several risks:

- AI may miss indirect module dependencies.
- It may select incomplete test cases.
- It may misunderstand the purpose of a code change.
- It may generate incorrect expected results.
- It may ignore critical business requirements.
- It may provide test selections without sufficient evidence.

Recommended Controls

1. Review AI-generated test selections manually.
2. Compare suggestions with the requirement traceability sheet.
3. Include mandatory critical tests in every release.
4. Verify the affected modules and service dependencies.
5. Store the final test selection and execution evidence.
6. Monitor missed defects and update the selection rules.
7. Avoid entering confidential passenger and payment information into unapproved AI tools.

AI should support the testing team rather than make final release decisions independently.

10.17 Conclusion for Task 10

The changed requirements significantly increase the complexity of the RailEase Railway Ticket Reservation System. The addition of mobile applications, UPI and wallet payments, a separate seat-recommendation service, 15,000 simultaneous users, fortnightly releases, and a smaller testing team requires an updated testing strategy.

The original balanced testing approach remains a suitable foundation, but it must be modified through automation, contract testing, performance testing, and staged rollout. Automated test execution and traceable reports can help the two-person testing team manage the shorter release cycle.

OVERALL CONCLUSION

The complete Software Testing Assignment provided a detailed understanding of different software testing levels and their practical application to the RailEase Railway Ticket Reservation System. The assignment covered unit testing, test case design, test harness construction, integration testing, system testing, regression testing, alpha testing, beta testing, formal acceptance testing, and integrated multi-level testing.

Through unit testing, the importance of verifying individual functions such as ticket fare calculation and refund calculation was understood. Different test design approaches showed how normal inputs, invalid inputs, and boundary conditions can be used to identify calculation

errors. The test harness activities demonstrated how external dependencies such as payment gateways can be simulated using stubs, mocks, and service virtualisation.

Integration testing highlighted the importance of communication between modules. The comparison of big-bang, incremental, and thread-based integration showed that testing complete business workflows helps identify interface problems more effectively. Integration test planning also demonstrated the importance of defining test scope, responsibilities, schedules, entry criteria, and exit criteria.

System testing focused on the complete application and its quality requirements. Functional testing verified the main booking operations, while performance testing examined response time and system capacity. Recovery, security, and compatibility testing helped evaluate how the application behaves under failures and across different environments.

Regression testing showed how test cases can be selected based on changes, historical faults, and business importance. Alpha, beta, and formal acceptance testing demonstrated the importance of involving internal users, regular travellers, and authorised business representatives before release.

The integrated testing activities showed that a balanced testing strategy provides better coverage than depending on only one testing level. Strong unit and integration testing help detect defects early, while system and acceptance testing verify complete application behaviour. The final adaptation task also demonstrated that testing strategies must change when software requirements, architecture, user load, release frequency, and team resources change.

Overall, the assignment strengthened the understanding that software testing is not simply the process of finding errors. It is a continuous activity that helps improve software reliability, performance, security, maintainability, and user satisfaction. For RailEase, a combination of planned testing, automation, risk-based regression, realistic performance evaluation, and formal acceptance provides a strong foundation for a reliable railway reservation system.